

TEMA 5

LAB PARALELISMO MULTITHREAD OPENMP



TEMA 5

LAB PARALELISMO MULTITHREAD

OPENMP

ARQUITECTURA DE COMPUTADORES

Dpto. Arquitectura de Computadores

Universidad de Málaga



CONTENIDOS

- Introducción
- Modelo de programación: memoria compartida
- OpenMP: Sintaxis
- OpenMP: constructs básicos

Transparencias basadas en el curso:

- Oscar Plata. Altas prestaciones para datos y ciberseguridad. Universidad de Málaga



INTRODUCCIÓN

TEMA 5: LAB PARALELISMO MULTITHREAD - OPENMP



INTRODUCCIÓN

- El objetivo de esta práctica es poner en práctica el conocimiento adquirido sobre el paralelismo multithread usando OpenMP
- Además, combinaremos paralelismo multithread y paralelismo de datos para obtener más rendimiento
- El ejemplo de partida es un código que calcula el **producto de dos matrices** cuadradas
 - Aparece en numerosas aplicaciones de diferentes ámbitos (aplicaciones gráficas, de simulación, científicas, etc.)
 - posee una estructura fácilmente extrapolable a otros códigos
- Usaremos **lenguaje de programación C**
 - Ejecutaremos sobre un sistema Linux



INTRODUCCIÓN

- OpenMP es uno de los modelos de programación paralela más utilizados en la actualidad (es relativamente fácil de usar)
- OpenMP → API para escribir aplicaciones multithread
 - Tres pilares: conjunto de directivas del compilador, biblioteca con rutinas para programar aplicaciones paralelas y variables de entorno
 - Simplifica enormemente la escritura de programas multithread en C/C++
 - Estándar para programación en entornos de memoria compartida
- Versión que soporta el compilador (gcc) en nuestro servidor docente: versión 4.5



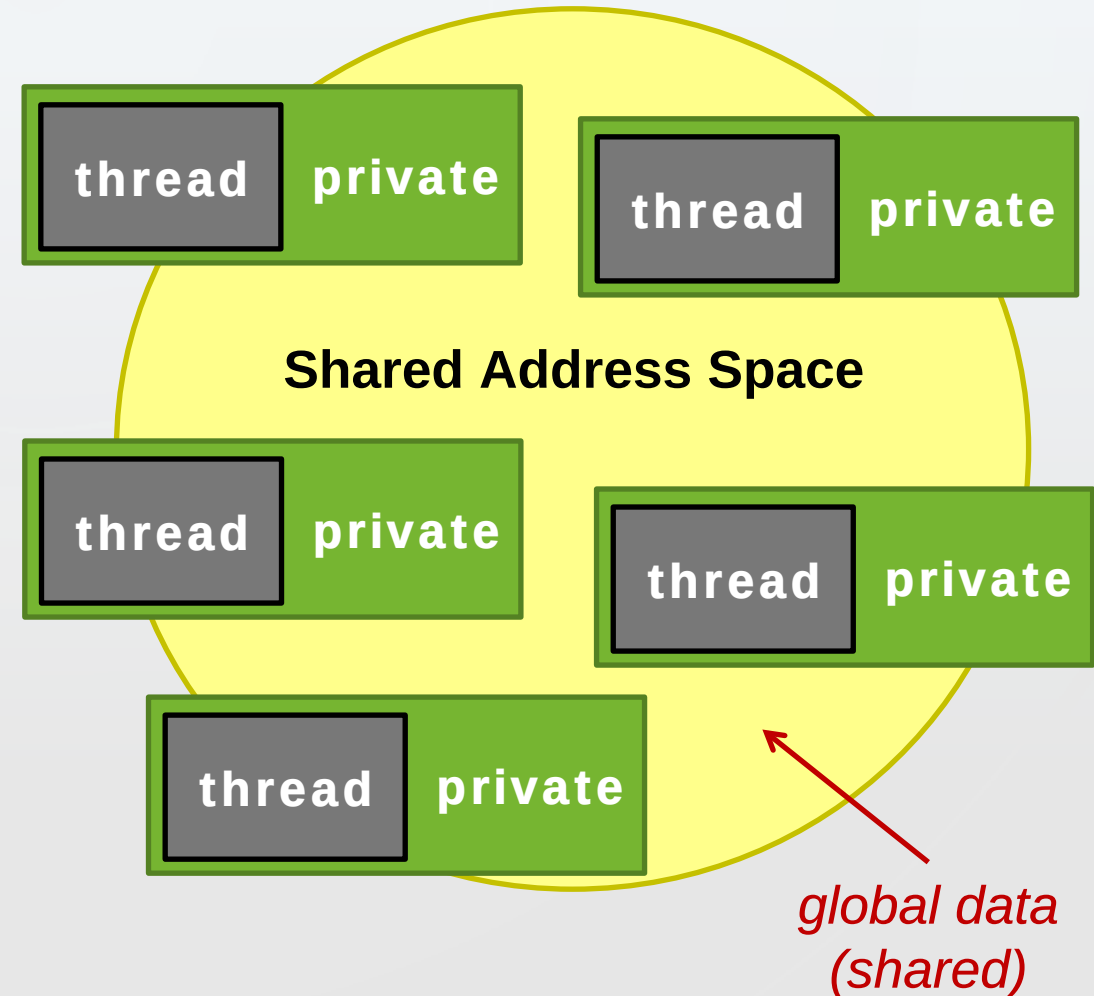
MODELO DE PROGRAMACIÓN: MEMORIA COMPARTIDA

TEMA 5: LAB PARALELISMO MULTITHREAD - OPENMP



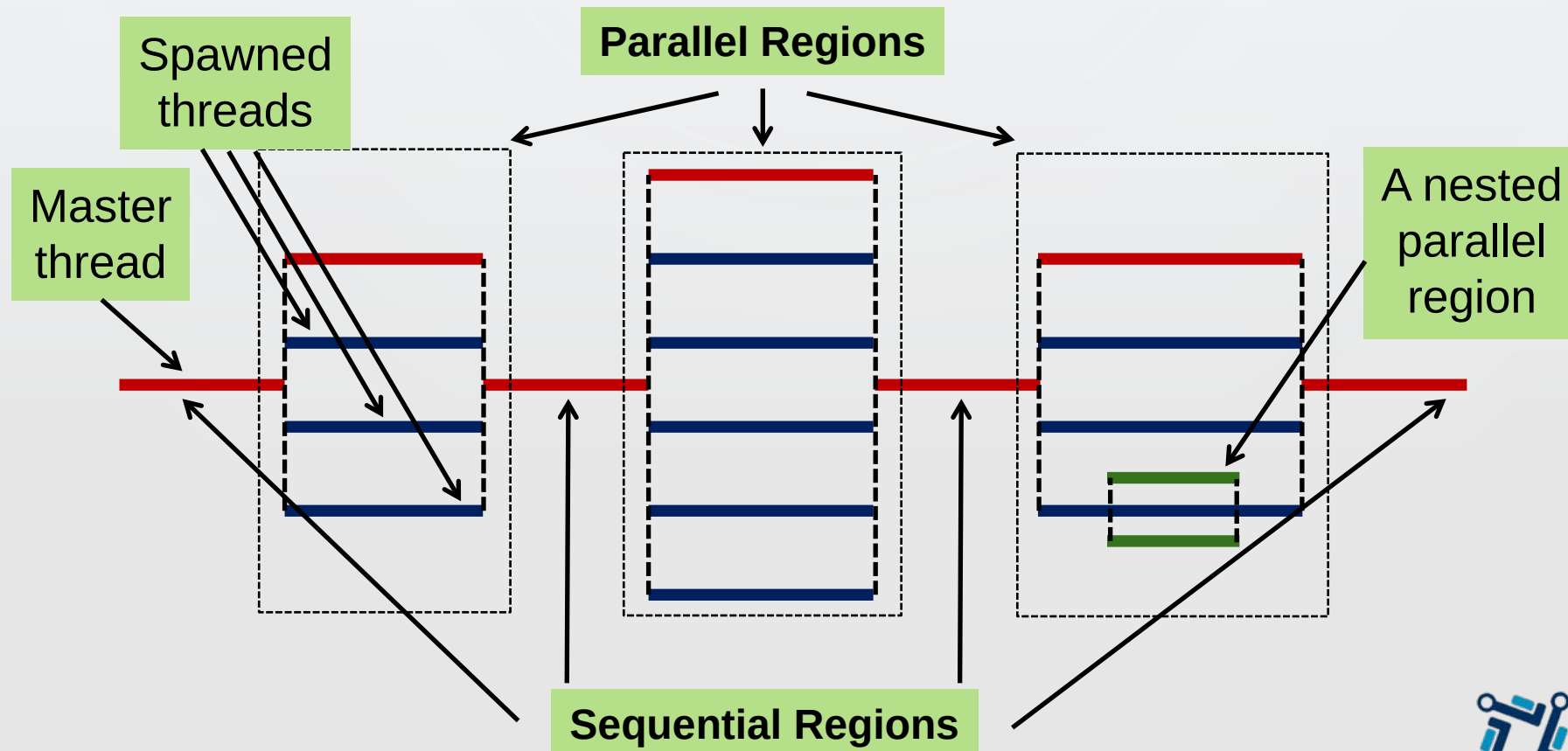
MEMORIA COMPARTIDA

- Instancia de un programa:
 - Un proceso y varios threads
 - Los threads interactúan en un mismo espacio de direcciones compartido a través de lecturas/escrituras
 - El planificador del SO decide cuándo ejecutar qué thread... entrelazados para garantizar la equidad (reparto equitativo de la carga de trabajo)
 - Sincronización para garantizar que la ejecución de las instrucciones den lugar a resultados correctos



OPENMP: MODELO DE PROGRAMACIÓN

- Modelo de paralelismo Fork-Join (diversificación-agrupamiento)
 - Al iniciar la ejecución de un programa, se inicia **un thread (master thread)**
 - El **master thread** se encarga de generar un **equipo de threads** según sea necesario
 - Paralelismo añadido incrementalmente hasta que se alcanzan los objetivos de rendimiento



OPENMP: SINTAXIS

TEMA 5: LAB PARALELISMO MULTITHREAD - OPENMP



NÚCLEO SINTAXIS EN OPENMP

- Prototipos de funciones y tipos declarados en `<omp.h>`
 - `#include <omp.h>`
- Se incluye en el código “constructs”. La mayoría de las constructs en OpenMP son directivas al compilador
 - `#pragma omp construct [clause [clause] ...]`
 - Example: `#pragma omp parallel num_threads(4)`
- La mayoría de las constructs de OpenMP se aplican a bloques estructurados
 - Un bloque estructurado es un bloque de una o más sentencias con un punto de entrada en la parte superior y un punto de salida en la parte inferior
 - Es OK tener un `exit()` dentro del bloque estructurado
 - En C, los bloques estructurados suelen estar delimitados por llaves (`{}`)



OPENMP: MODELO DE PROGRAMACIÓN

- Directivas del compilador:
 - `#pragma omp construct [clause [clause] ...]`
- Variables de entorno:
 - `OMP_NUM_THREADS N` (set the number of threads)
 - `OMP_DYNAMIC true | false` (set the number of threads automatically)
 - `OMP_SCHEDULE schedule[,chunk]` (iteration scheduling scheme)
- Biblioteca de rutinas:
 - `omp_get_thread_num()` (get thread ID)
 - `omp_get_num_threads()` (get number of threads)
 - `omp_set_num_threads(int n)` (set the number of threads)
 - `omp_get_wtime()` (returns elapsed wallclock time)



EJEMPLO SIMPLE

- Programa Secuencial

- Un proceso con un único thread

```
int main()
{

    int ID=0;
    printf("hello(%d)", ID);
    printf("world(%d)\n", ID);

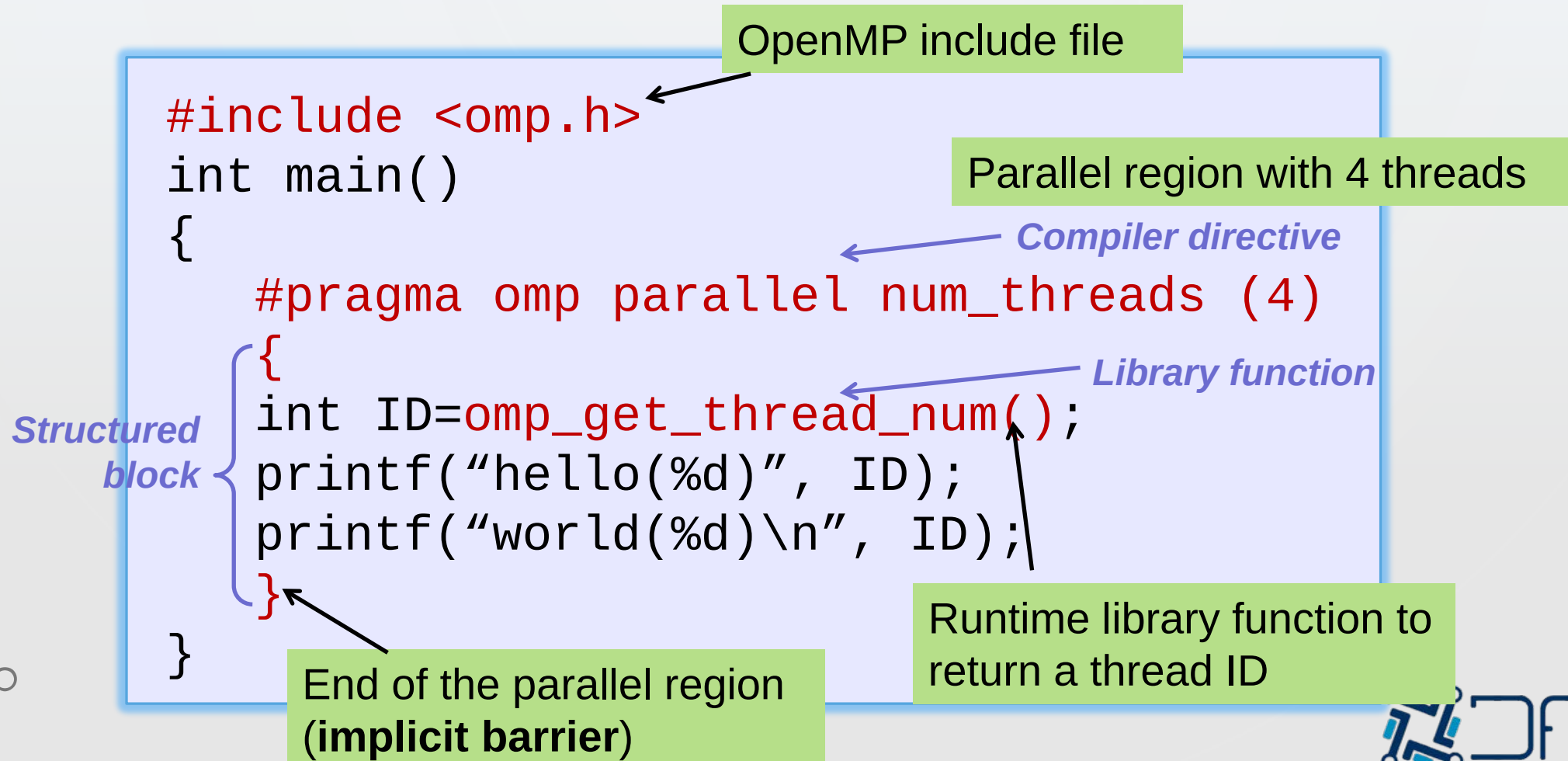
}
```



EJEMPLO SIMPLE

- Programa multithread

- Un proceso con múltiples threads



EJEMPLO SIMPLE

- Programa multithread

- Un proceso con múltiples threads

Each thread executes a copy of the code within the structured block

3 additional threads are created here (the master thread is the 4th one)

```
#include <omp.h>
int main()
{
```

```
#pragma omp parallel num_threads (4)
```

```
{
  int ID=omp_get_thread_num();
  printf("hello(%d)", ID);
  printf("world(%d)\n", ID);
}
```

```
}
```

Parallel directive requests 4 threads

The 3 additional threads are killed here (the master thread continues alone)

Runtime function returning a thread ID (ID=0,1,2,3)



COMPILACIÓN Y EJECUCIÓN CON OPENMP

- En Linux con gcc

- gcc -fopenmp prog.c -o prog

- export OMP_NUM_THREADS=4 (Bash shell)

- ./prog

Variable de entorno

- El programa OpenMP (prog) se ejecuta como un único proceso, con múltiples threads (multithread)



OPENMP: CONSTRUCTS BÁSICOS

TEMA 5: LAB PARALELISMO MULTITHREAD - OPENMP



PRINCIPALES CONSTRUCTS DE OPENMP

- Para crear un equipo de threads:
 - `#pragma omp parallel`
- Para distribuir la carga entre los threads:
 - `#pragma omp for`
 - `#pragma omp sections`
 - `#pragma omp single,`
`#pragma omp master`
 - `#pragma omp task`
- Para prevenir conflicto de datos (sincronización):
 - `#pragma omp critical`
 - `#pragma omp barrier`
- Atributos (clauses) para la compartición de datos:
 - `shared(var1, var2...)`
 - `private(var1, var2...)`
 - `firstprivate(var1, var2...)`
 - `lastprivate(var1, var2...)`
 - `reduction(op:var1, var2...)`



PARALLEL VS. WORKSHARING CONSTRUCTS

- Un **parallel construct** crea un programa multithreaded donde cada thread ejecuta redundantemente el mismo código
 - `#pragma omp parallel`
- Un **worksharing construct** reparte la carga de trabajo del código entre los threads de un equipo
 - Loop construct
 - Section construct
 - Single construct
 - Task construct
- Pero ambas tienen una barrera implícita en la salida pero no en la entrada



COMBINACIÓN PARALLEL /WORKSHARING

- Atajo en OpenMP: se pueden combinar ambas constructs en una misma linea de código

```
double res[MAX];  
int i;  
  
#pragma omp parallel  
{  
    #pragma omp for  
    for (i=0; i<MAX, i++){  
        res[i] = compute(i);  
    }  
}
```

```
double res[MAX];  
int i;  
  
#pragma omp parallel for  
    for (i=0; i<MAX, i++){  
        res[i] = compute(i);  
    }
```



SINCRONIZACIÓN: BARRIER

- Cada thread espera hasta la llegada de todos los threads

```
#include <omp.h>
int main()
{
    #pragma omp parallel
    {
        int ID=omp_get_thread_num();
        A[ID] = calc1(ID);
        #pragma omp barrier
        B[ID] = calc2(ID,A);
    }
}
```

No threads compute
calc2() until all threads
have computed calc1()



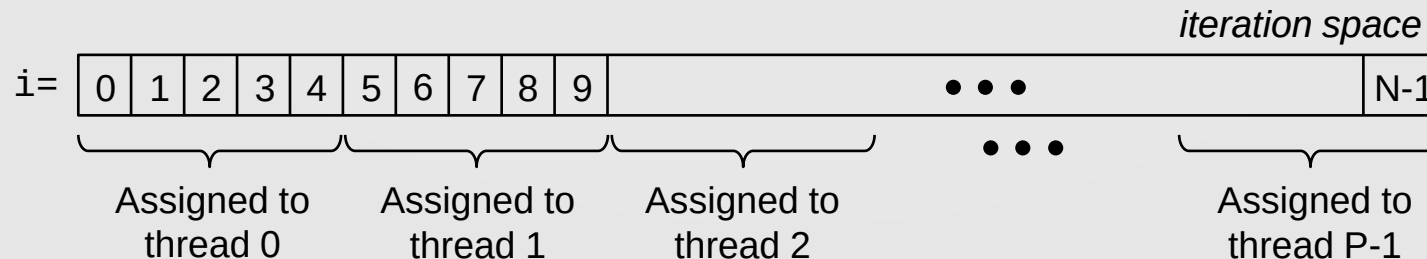
LOOP WORKSHARING CONSTRUCT

- El loop worksharing construct divide el número de iteraciones de un bucle entre los threads de un equipo

Variable `i` is made private to each thread by default

```
#pragma omp parallel
{
    #pragma omp for
    for (i=0; i<N, i++){
        compute(i);
    }
}
```

- Por defecto, el espacio de iteraciones de un bucle es dividido en P (siendo P el número de threads), asignando cada bloque a un thread



LOOP WORKSHARING CONSTRUCT: CLAUSE

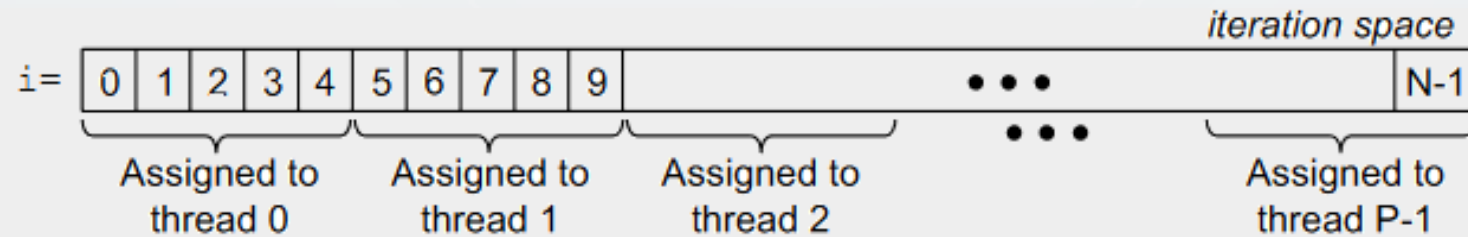
- Schedule clause influye en como se mapean las iteraciones del bucle en los threads (reparto de la carga)
 - `schedule(static)`: la planificación por defecto (un bloque de iteraciones por thread)
 - `schedule(static [, chunk] )` : reparte bloques de iteraciones de tamaño "chunk" a cada thread
 - `schedule(dynamic [, chunk] )` : cada thread coge de forma dinámica un "chunk" de iteraciones de una cola



LOOP WORKSHARING CONSTRUCT: CLAUSE

- Ejemplos

`schedule(static)` (distribución de bloques)



`schedule(static, 1)` (distribución cíclica o entrelazada)

